Final Project: Financial News Sentiment & FDIC Banking Stress

Table of contents

Project Information

- Group members: [TODO]
- Lecture section time: [TODO]
- GitHub usernames: [TODO]

Overview

This project asks:

1. Can LLM-derived sentiment signals from financial news predict deteriorations in state-level banking fundamentals (FDIC)?

2. Do sentiment indicators provide earlier or stronger signals than traditional FDIC fundamentals (e.g., ROA/NIM)?
3. Are stress reactions geographically concentrated across US states?

Research Questions

This project studies whether regulatory-finance news sentiment contains useful early warning signals for banking stress at the US state-year level.

1. **Predictive signal**: Is a higher share of negative regulatory-news sentiment associated with worse banking outcomes (especially lower next-period ROA)?
2. **Incremental value**: Does sentiment improve stress identification when used together with FDIC fundamentals such as lagged ROA, NIM, and balance-sheet growth?
3. **Spatial concentration**: Are model-implied stress levels clustered in particular states, suggesting uneven geographic vulnerability?

Data

- News/regulatory text: `raw_partner_headlines.csv` (filtered to regulatory keywords; scored with a lightweight sentiment model)
- Banking fundamentals: FDIC state-year aggregates (e.g., `Summary_data_states.csv`)
- Geography: US state boundaries (Census shapefile; used for choropleth map)

Folder conventions

- Raw inputs go in `data/raw-data/`
- Derived/clean tables go in `data/derived-data/`

Data & Methods

All of the processing, merging, and reshaping for this project is handled directly in this `.qmd` file via Python code chunks. The helper module `preprocessing.py` exposes reusable functions (for presentation slides and the Streamlit app), but the core wrangling logic is executed here.

Method summary:

1. Load FDIC state-year data and construct derived features such as `ROA`, `DROA`, lagged controls, and stress labels (`bad_year`, `severity`).
2. Train a lightweight TF-IDF + logistic-regression sentiment model using the labeled financial text corpus.
3. Score filtered regulatory-news headlines and aggregate sentiment to the year level.

4. Merge yearly sentiment with FDIC panel data.
5. Fit baseline classification and severity models, then combine them into `StressScore = p_bad_year * sev_hat`.

```
import numpy as np
import pandas as pd
from pathlib import Path

import preprocessing as prep

# In Quarto/Jupyter execution, __file__ is usually undefined.
# Use current working directory as repository root.
REPO_ROOT = Path.cwd()
RAW_DIR = REPO_ROOT / "data" / "raw-data"
DERIVED_DIR = REPO_ROOT / "data" / "derived-data"
```

Build state–year panel from raw data

Load FDIC state–year aggregates from `Summary_data_states.csv` and construct ROA, ΔROA, and related fundamentals.

```
fdic = prep.load_fdic_state_year()
fdic[["STNAME", "YEAR", "ROA", "DROA"]].head()
```

	STNAME	YEAR	ROA	DROA
1	Alabama	1934	0.0	NaN
60	Alabama	1935	0.0	0.0
119	Alabama	1936	0.0	0.0
178	Alabama	1937	0.0	0.0
237	Alabama	1938	0.0	0.0

Train a lightweight sentiment model on the labeled news dataset and aggregate headline sentiment to the year level. To make this chunk self-contained (so it works even if run independently), we reload the FDIC panel to determine the relevant year range.

```
# Load FDIC state-year data to determine year range
fdic = prep.load_fdic_state_year()

# Train text classifier on labeled news (all-data.csv)
vec, clf = prep.train_sentiment_model()
```

```python
# Limit to the same time window used in the presentation/app
min_year = int(fdic["YEAR"].max() - 10) if not fdic.empty else 2010
max_year = int(fdic["YEAR"].max()) if not fdic.empty else 2024

keywords = ["fdic", "sec", "fed", "regulation", "bank run", "regional bank", "stress test"]
yearly_sent = prep.build_news_sentiment_yearly(
    vec,
    clf,
    keywords=keywords,
    min_year=min_year,
    max_year=max_year,
)
yearly_sent.head()
```

YEAR	sent_mean	sent_neg_share	news_count

Merge the FDIC panel with the yearly sentiment index and fit simple models to construct `StressScore`, which summarizes both the probability and severity of bad banking years.

```python
panel = fdic.merge(yearly_sent, on="YEAR", how="left")
panel = prep.fit_and_score_panel(panel)

panel[["STNAME", "YEAR", "ROA", "DROA", "bad_year", "sent_neg_share", "StressScore"]].head()
```

	STNAME	YEAR	ROA	DROA	bad_year	sent_neg_share	StressScore
0	Alabama	1934	0.0	NaN	0	NaN	NaN
1	Alabama	1935	0.0	0.0	0	NaN	NaN
2	Alabama	1936	0.0	0.0	0	NaN	NaN
3	Alabama	1937	0.0	0.0	0	NaN	NaN
4	Alabama	1938	0.0	0.0	0	NaN	NaN

For faster iteration (e.g., when developing the Streamlit app), you can also run the standalone preprocessing script, which writes the same derived panel and static assets to disk:

```
python preprocessing.py
```

Results

Static Plot 1: Sentiment vs ΔROA scatter

We first show how regulatory-news negativity relates to changes in ROA at the state–year level using an Altair scatter plot.

```
import altair as alt

latest_year = int(panel["YEAR"].dropna().max())
scatter_df = panel.dropna(subset=["sent_neg_share", "DROA"]).copy()
scatter_df = scatter_df[scatter_df["YEAR"].between(latest_year - 5, latest_year)]

alt.Chart(scatter_df).mark_circle(size=60, opacity=0.4).encode(
    x=alt.X("sent_neg_share:Q", title="Negative probability (yearly mean)"),
    y=alt.Y("DROA:Q", title="ΔROA"),
    color=alt.Color("bad_year:N", title="Bad year"),
    tooltip=["STNAME:N", "YEAR:Q", "ROA:Q", "DROA:Q", "sent_neg_share:Q", "StressScore:Q"],
).properties(
    height=360,
    width=600,
    title="News negativity vs next-period ROA change (state-year)",
)
```

```
alt.Chart(...)
```

Interpretation: points with higher `sent_neg_share` are expected to align with weaker `DROA` values if sentiment captures stress conditions. This plot provides a direct visual check for Research Questions 1 and 2 by showing whether more negative news is associated with poorer next-period profitability outcomes.

Static Plot 2 (Spatial): US map of stress indicators

Next we map the StressScore across US states using Altair. We focus on the most recent year available in the panel.

```
STATE_FIPS = {
    "Alabama": 1, "Alaska": 2, "Arizona": 4, "Arkansas": 5, "California": 6, "Colorado": 8,
    "Connecticut": 9, "Delaware": 10, "District of Columbia": 11, "Florida": 12, "Georgia": 1
    "Hawaii": 15, "Idaho": 16, "Illinois": 17, "Indiana": 18, "Iowa": 19, "Kansas": 20,
    "Kentucky": 21, "Louisiana": 22, "Maine": 23, "Maryland": 24, "Massachusetts": 25,
```

```
    "Michigan": 26, "Minnesota": 27, "Mississippi": 28, "Missouri": 29, "Montana": 30,
    "Nebraska": 31, "Nevada": 32, "New Hampshire": 33, "New Jersey": 34, "New Mexico": 35,
    "New York": 36, "North Carolina": 37, "North Dakota": 38, "Ohio": 39, "Oklahoma": 40,
    "Oregon": 41, "Pennsylvania": 42, "Rhode Island": 44, "South Carolina": 45,
    "South Dakota": 46, "Tennessee": 47, "Texas": 48, "Utah": 49, "Vermont": 50,
    "Virginia": 51, "Washington": 53, "West Virginia": 54, "Wisconsin": 55, "Wyoming": 56
}

map_year = latest_year
map_df = panel[(panel["YEAR"] == map_year)].dropna(subset=["StressScore"]).copy()
map_df["STNAME"] = map_df["STNAME"].astype(str).str.strip()
map_df["id"] = map_df["STNAME"].map(STATE_FIPS)
map_df = map_df.dropna(subset=["id"]).copy()
map_df["id"] = map_df["id"].astype(int)

us_states = alt.topo_feature(
    "https://cdn.jsdelivr.net/npm/vega-datasets@v1.29.0/data/us-10m.json",
    "states",
)

background = alt.Chart(us_states).mark_geoshape(
    fill="lightgray",
    stroke="white",
)

foreground = alt.Chart(us_states).mark_geoshape(stroke="white").transform_lookup(
    lookup="id",
    from_=alt.LookupData(map_df, "id", ["STNAME", "StressScore"]),
).encode(
    color=alt.Color("StressScore:Q", title="StressScore", scale=alt.Scale(scheme="orangered")
    tooltip=["STNAME:N", "StressScore:Q"],
)

(background + foreground).project(type="albersUsa").properties(
    width=700,
    height=430,
    title=f"Geographic distribution of StressScore, {map_year}",
)
```

```
alt.LayerChart(...)
```

Interpretation: darker states indicate higher model-implied stress. This choropleth addresses

Research Question 3 by showing whether stress risk appears geographically concentrated rather than evenly distributed across states.

Streamlit App

The Streamlit app lives in `streamlit-app/`. See `README.md` for how to run it locally (deployment/link details: [TODO]).

The app is designed to support the writeup in three ways:

- **Exploration**: users can inspect sentiment and banking-stress indicators interactively instead of relying only on static snapshots.
- **Cross-checking**: users can compare patterns seen in the static figures with additional years or states in an interactive view.
- **Communication**: the app provides an accessible interface for explaining how sentiment features and FDIC fundamentals jointly inform the stress signal.

Limitations

- The headline source used for yearly sentiment may be incomplete in this repository snapshot; when missing, sentiment aggregates are unavailable.
- The modeling strategy is intentionally lightweight and does not establish causal effects between sentiment and banking outcomes.
- State-year aggregation can hide within-state bank heterogeneity and timing effects at higher frequency (quarterly/monthly).